FLIGHT CONTROL AND 3D VISUALIZATION OF CRAZYFLIE 2.1

Amna Ahmad (EE-19)
Jawad Ali Abbasi (EE -19)

SUPERVISOR
DR. RAHEEL QURAISHI

NATIONAL ENGINEERING AND SCIENTIFIC COMMISSION

(NESCOM)


بسم الله الرحمن الرحيم

DECLARATION

I hereby declare that this report is entirely based on my personal efforts and learning made under the sincere guidance of my supervisor. If any part of this report is proved to be copied out or found to be reported, I shall stand by the consequences. No portion of the work presentedin this report has been submitted in support of any application for any other institute of learning.

Date: _____

Amna Ahmad (EE-19)

Jawad Ali Abbasi (EE -19)

Contents

FLIGHT CONTROL AND 3D VISUALIZATION OF CRAZYFLIE 2.1 1
DECLARATION 2
ACKNOWLEDGEMENTS 5
ABSTRACT 6
INTRODUCTION 7
Hardware overview: 7
1.Assembly 7
1.1 Components 7
1.2 Self-test 1: 8
1.3 Assembling step 1 8
1.4 Assembling step 2: 9
1.5 Assembling step 3: 9
1.6 Assembling step 4: 9
1.7 Assembling step 5: 10
1.8 Power test 2: 10
2.Flight control with Bluetooth 10
2.1 Procedure: 10
2 .2 Crazyflie's app user interface: 10
2.3 Observation 11
3 .Flight control with CF-client: 11
3.1 Introduction: 11
3.2 Installation: 11
3.3 CF-Client user interface: 11
3.4 GUI overview [6] 12
3.5 Tabs and toolboxes: 12
3.6 Observation: 12
4 Drone flight simulation using MATLAB: 13
4.1 Explanation 13
4.2 Parameters: 13
4.3 Schematic model: 14
4.4 Input signals 14
4.5 Output simulation: 15
5. Loco Positioning System using Anchor Node Communication 15
5.1 Introduction: 15
5.2 Explanation of working: 15
5.3 Methods of loco positioning: 16

5.4 Software requirements:.....16
5.5 Steps for setting up system: 16
5.6 Experimental results.....17
6.Ros2 (RVIZ2) (Connections + Readings) 18
6.1 Software requirments: 18
6.2 Explanation:.....18
6.3 ROS 2 configurations: 19
6.4 Challenges: 20
7.Flight control with Python API 20
7.1 Software installation:.....20
7.2Connecting, Logging and Parameters:.....20
7.3Step 1. Connecting with the crazyflie:.....20
7.4Step 2.Takeoff+deck_Detection:.....21
7.5Step 2.1 Logging Synchronous:.....23
7.7 Step 3. Logging Asynchronous:.....25
7.8 Step 4. Parameter retrieve +update 26
7.9 Step 5. Flying + Motion Commander + Loggings + Graphs:.....28
7.10 Step 6. Flying in a box + Motion Commander + Loggings Graphs: 31

ACKNOWLEDGEMENTS

We extend our heartfelt gratitude to the Almighty, Allah, for granting us the willpower, strength, and resources to successfully complete this internship. Without His grace, this endeavor would have remained unrealized. May His blessings be upon the Holy Prophet Muhammad (PBUH), whose teachings have led us from darkness into the light

We sincerely appreciate our supervisor, Dr. Raheel Quraishi, for his guidance, and support, which have enriched our internship experience and fueled our determination to excel in the tasks assigned.

We would also like to extend our thanks to NESCOM for their generous provision of equipment and resources, which played a pivotal role in the successful execution of our internship project. Their support has been instrumental in achieving our goals.

Furthermore, we are indebted to our university, IST (Institute of Space Technology), for granting us access to their state-of-the-art laboratories, which were essential in conducting our project. Their support and commitment to fostering educational and research endeavors are greatly appreciated.

we dedicate this achievement to our parents, whose unwavering support have fueled our journey. Their sacrifices and guidance have shaped us, and we carry their blessings with deep gratitude.

In conclusion, the successful completion of this internship has been a collective effort, and we are sincerely thankful to all who contributed, directly or indirectly, to this achievement.

Amna Ahmad

Jawad Ali Abbasi

ABSTRACT

During our internship at the National Engineering and Scientific Commission, our primary focus was on advancing our drone proficiency through software interfacing and achieving precise flight control. Our journey began with the assembly of the Crazyflie mini quadcopter drone, meticulously ensuring the proper functioning of all hardware components.

In our initial flight tests, we established a Bluetooth connection between the drone and our mobile device to monitor its performance and ensure error-free flight.

Subsequently, we delved into interfacing the drone with the CF Client software provided by Crazyflie. Here, we executed fundamental commands for takeoff and landing while closely monitoring various drone parameters.

To gain deeper insight into the internal workings of the drone, we utilized general transfer functions and parameters in MATLAB. This enabled us to use flight simulations, allowing us to observe the drone's behavior when subjected to different flight signals.

Our next step involved using Python to observe parameter logs and take control of the drone's flight by transmitting compiled code to the STM32 microcontroller via radio communication. Additionally, we utilized Matplotlib libraries to generate plots illustrating the drone's x, y, and z coordinates during flights.

To enhance our capabilities further, we were equipped with the necessary tools for local positioning. This technology allowed us to create a precise 3D representation of a room, strategically positioning nodes to provide real-time, accurate tracking of the drone's location. These nodes communicated with the deck attached to the drone, effectively creating a mini GPS system within the room. Using the local positioning feature in the CF Client, we could visualize and fine-tune this setup.

As a culmination of our efforts, we decided to produce a similar 3D representation of both the drone and radio communication using Rviz in Ross2, further expanding our skills and understanding of drone technology.

INTRODUCTION

The Crazyflie 2.1 is an incredible open-source flying platform designed to offer students the opportunity to explore the fascinating realm of drone technology. This versatile open-source flying development platform, weighing a mere 27g, has become a valuable asset for countless students embarking on projects that require a dynamic and user-friendly drone. the Crazyflie 2.1 is also equipped with low-latency/long-range radio as well as Bluetooth LE. It is also compatible with an extensive ecosystem of software and deck expansions it's ideal for education, research and development.[1]

Hardware overview:

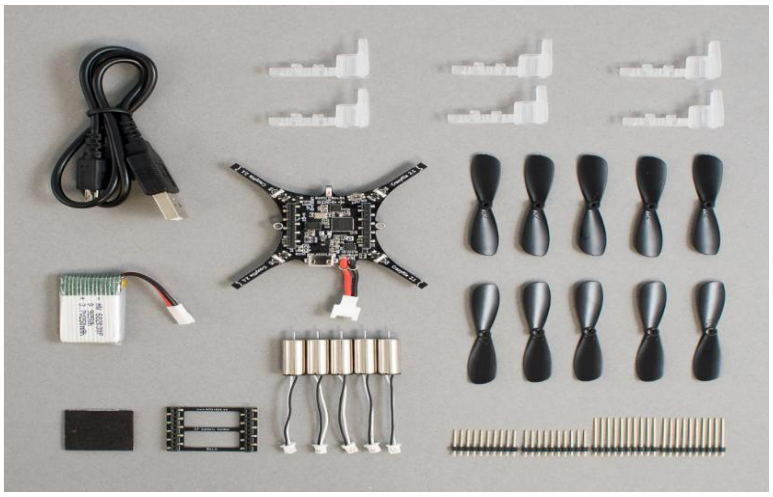
1. **Motor and Propellers:** The Crazyflie 2.1 is equipped with four coreless DC motors, each driving a propeller. These motors provide thrust for lift and manoeuvrability.
2. **Flight Controller Board:** At the heart of the Crazyflie 2.1 is the flight controller board. This board houses a powerful microcontroller that processes data from an array of sensors, including a gyroscope, accelerometer, and magnetometer.
    - **Gyroscope:** The gyroscope sensor measures angular velocity, helping the drone maintain stability and orientation during flight.
    - **Accelerometer:** The accelerometer measures linear acceleration, enabling the drone to detect changes in speed and direction.
    - **Magnetometer:** The magnetometer senses magnetic fields, aiding in navigation and heading control.
3. **Battery Compartment:** The battery compartment houses the rechargeable Li-Po (Lithium Polymer) battery that powers the drone. It's typically located underneath the main body.
4. **Radio Transceiver:** The Crazyflie 2.1 features a radio transceiver that allows it to communicate wirelessly with a controlling device, such as a computer, smartphone, or remote control.
5. **LEDs (Light Emitting Diodes):** LED indicators are often integrated into the drone's design. They serve multiple purposes, including indicating the drone's status, battery level, and orientation.

1. Assembly

1.1 Components

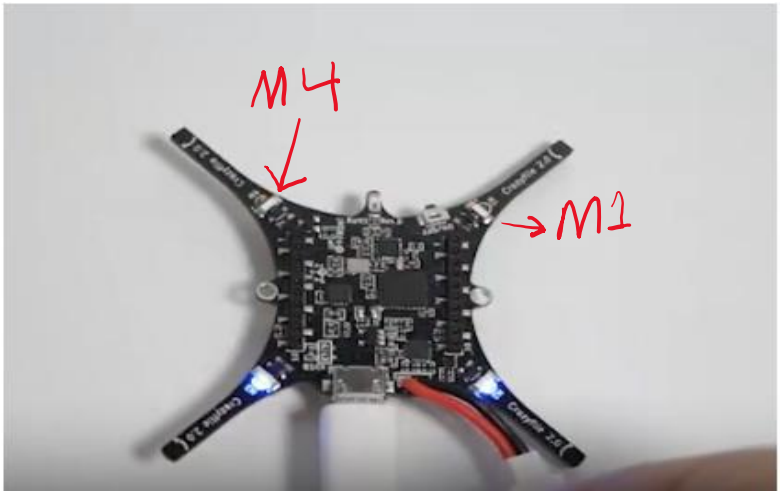
- 1 x Crazyflie 2.X control board with all components mounted
- 5 x CW propellers
- 5 x CCW propellers
- 6 x Motor mounts
- 1 x LiPo battery

- 5 x Coreless DC motors
- 2 x Short expansion connector pins (1×10, 2mm spacing, 8 mm long)
- 2 x Long expansion connector pins (1×10, 2mm spacing, 14 mm long)
- 1 x Battery holder expansion board
- 1 x USB cable (only with the Crazyflie 2.1)



1.2 Self-test 1:

- run the power on self-test by connecting the Crazyflie 2.X to a uUSB power source. The LEDs M1 and M4 will indicate the result of the test. If the M4 LED blinks GREEN five times fast, then the test has passed. If the self-test fails, then the M1 LED blinks 5 times fast RED, then pauses and does it again[2]



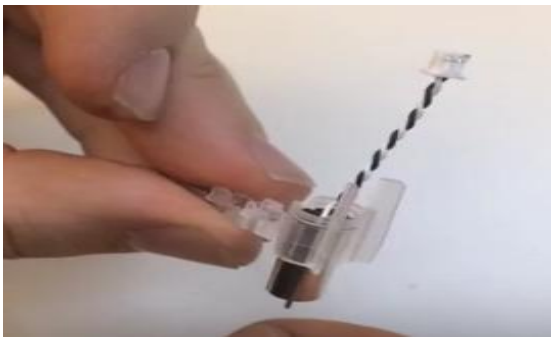
1.3 Assembling step 1

- Start by twisting the wires of the four motors. This will reduce electronic noise and make the wires fit better in the motor mount “hooks”.



1.4 Assembling step 2:

- Push the four motors into the motor mounts. You will need some force to insert them. The motor should be inserted all the way to the stop in the mount. Attach the twisted wire into the two small “hooks” that are underneath the motor mount.

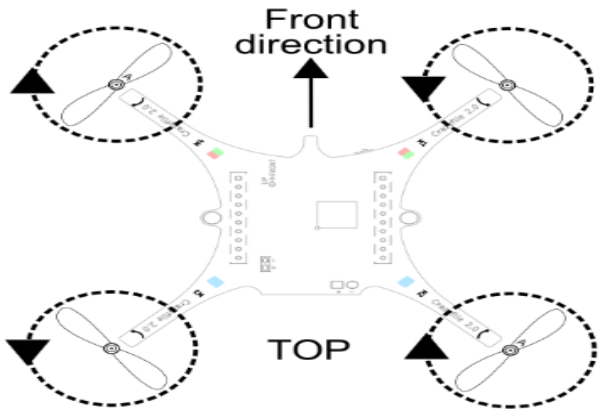


1.5 Assembling step 3:

- Insert the motor mounts on the Crazyflie 2.X wings. They are press fit and might need a small amount of force. Make sure they go all the way to the stop. It’s not important which motor you put where. After it’s been inserted, connect the motor connectors to the Crazyflie 2.X.

1.6 Assembling step 4:

- There are two kinds of propellers, the clock wise (CW) and counter clock wise (CCW) propellers, each kind has their own bag in the box. Notice the shape of the tips, the sharper corner is on back side of the rotation direction. The CW propellers are also usually marked with an “A”, “A1” or “A2”, while the CCW propellers are marked with “B”, “B1” or “B2” (the number is irrelevant).
- The correct side should be facing up, the top side should be convex.



1.7 Assembling step 5:

- The rubber pad should be attached to the Crazyflie 2.X between the expansion headers. This will create friction, keep the battery from slipping out and also protect the electronics. Attach the headers. Place the battery between the headers inserted into the expansion connector and insert the battery holder board onto the headers.

1.8 Power test 2:

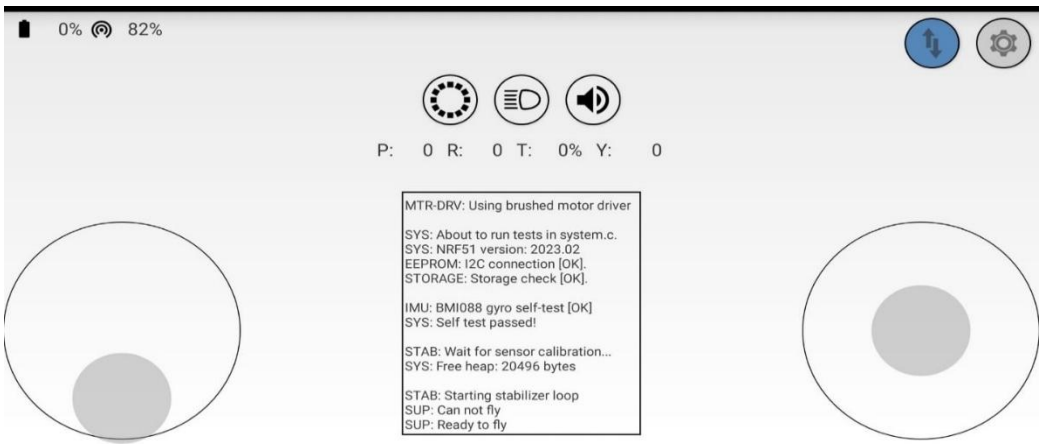
- During the power-on self-test all the propellers will spin in sequence. Make sure they all spin, if they don't then check the motor connections.

2. Flight control with Bluetooth

2.1 Procedure:

We installed the crazyflie app on our phone. Then, we used Bluetooth to link the app[4] with our drone. We used the app to make the crazyflie fly around and control it. However, the crazyflie has a limited range so we tested it out in a small area. The main purpose of doing this was to ensure that there were no bugs or errors in the drone's flight

2 .2 Crazyflie's app user interface:



2.3 Observation:

While only giving thrust to the drone it would move upwards and gain height proportional to the level of thrust provided through the controller however it would also move forward due to an uneven balance of the drone's weight

3 .Flight control with CF-client:

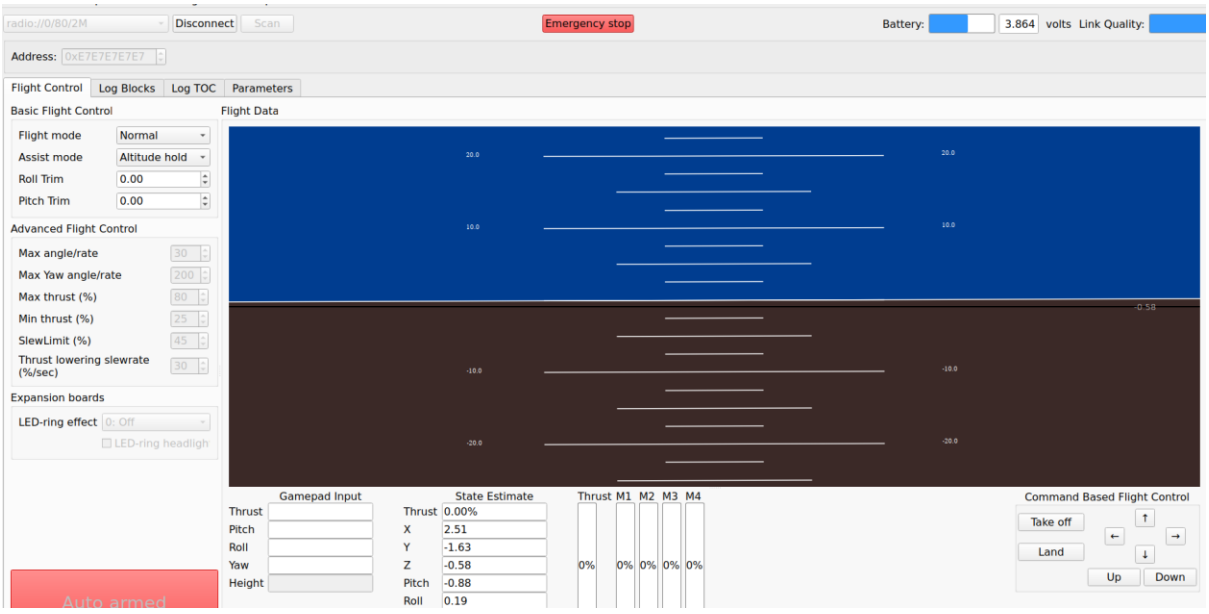
3.1 Introduction:

CF-Client, short for Crazyflie Client[5], is a powerful software tool designed specifically for interacting with and controlling Crazyflie drones. This versatile application serves as the interface between users and their Crazyflie drones, offering a user-friendly platform for configuring, monitoring, and piloting these agile flying machines.

3.2 Installation:

1. Install Python version is 3.7+ and Ubuntu 20.04 system,
2. For running the client form source install git, pip and a lib for the Qt GUI.
3. On terminal of Ubuntu/Linux run following commands:
4. sudo apt install git python3-pip libxcb-xinerama0
5. pip3 install --upgrade pip
6. git clone https://github.com/bitcraze/crazyflie-clients-python
7. cd crazyflie-clients-python
8. $ pip3 install -e .
9. $ pip3 install -e .[dev]

3.3 CF-Client user interface:



3.4 GUI overview [6]:

1. The window title will show the connection status
2. Connect/disconnect, scan and the drop-down connection list as well as Address and auto reconnect.
3. Scan: Will scan for available Crazyflies within the chosen address.
4. Connect: Will connect to the selected URI in the connection list
5. Disconnect: Will disconnect the current Crazyflie
6. Address: The address to scan for. If you didn't change this in the configuration, then leave the default of 0xE7E7E7E7E7
7. Battery and link quality (from 0% to 100%)
8. Tabs with specific functionality (see below for details)
9. The content for the active tab
10. Toolboxes can be docked to the edges, making their content visible all the time.
11. The selected and used input interface
12. Emergency stop button. The motors will stop when clicked.

3.5 Tabs and toolboxes:

To open a toolbox, use the View->Toolboxes menu, if the toolbox is already open as a tab, the tab will be removed and it will be opened as a toolbox instead.

The following tabs/toolboxes are currently available:

1. Flight control Tab
2. Console Tab
3. Plotter Tab
4. Parameter Tab
5. Log Blocks Tab
6. Lighthouse Positioning Tab
7. Loco Positioning Tab
8. Tuning Tab

3.6 Observation: All parameters like thrust pitch and roll could be observed. Flight command in the bottom right conner were used[6]

Takeoff: this command would give thrust and the drone would fly north

Land: this command would stop the propellers and the drone would lose the height it gained

4 Drone flight simulation using MATLAB:

4.1 Explanation:

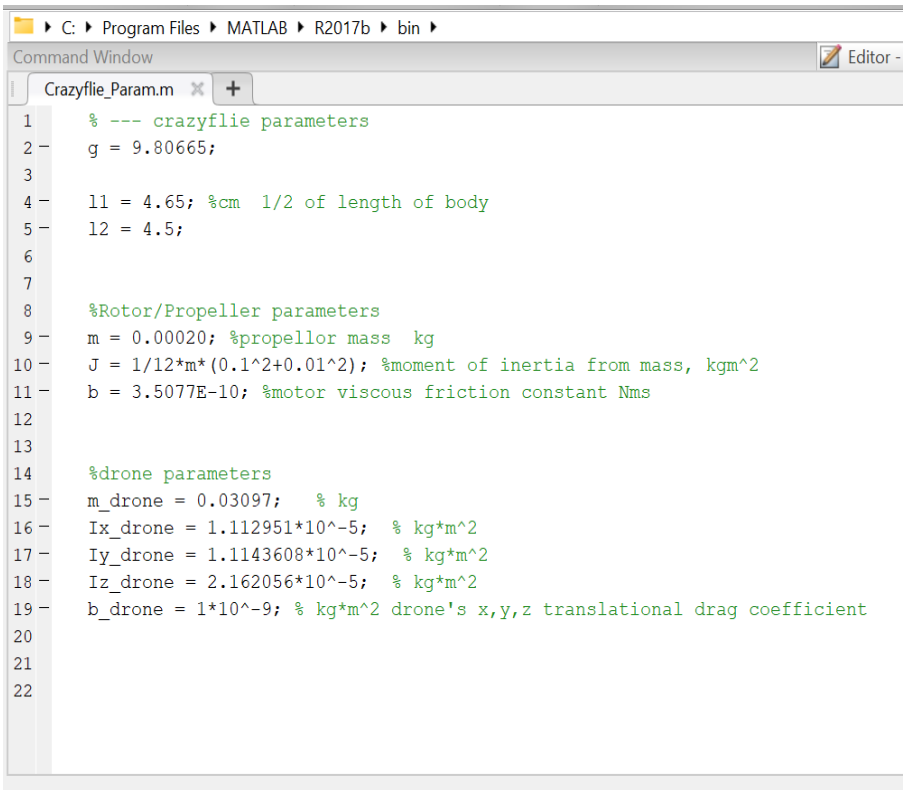
With the help of the research done by Wookshik Kim [3] we used the estimated parameters in a separate .m file and used the schematic diagram in Simulink to run a 3d visual trajectory of the drone when different input signals were sent to it

The attitude controller employs a nested PID controller integrated into the Crazyflie firmware. It takes PWM signals as input, with a maximum value of 65535, corresponding to 16 bits, consistent with the Crazyflie Firmware.

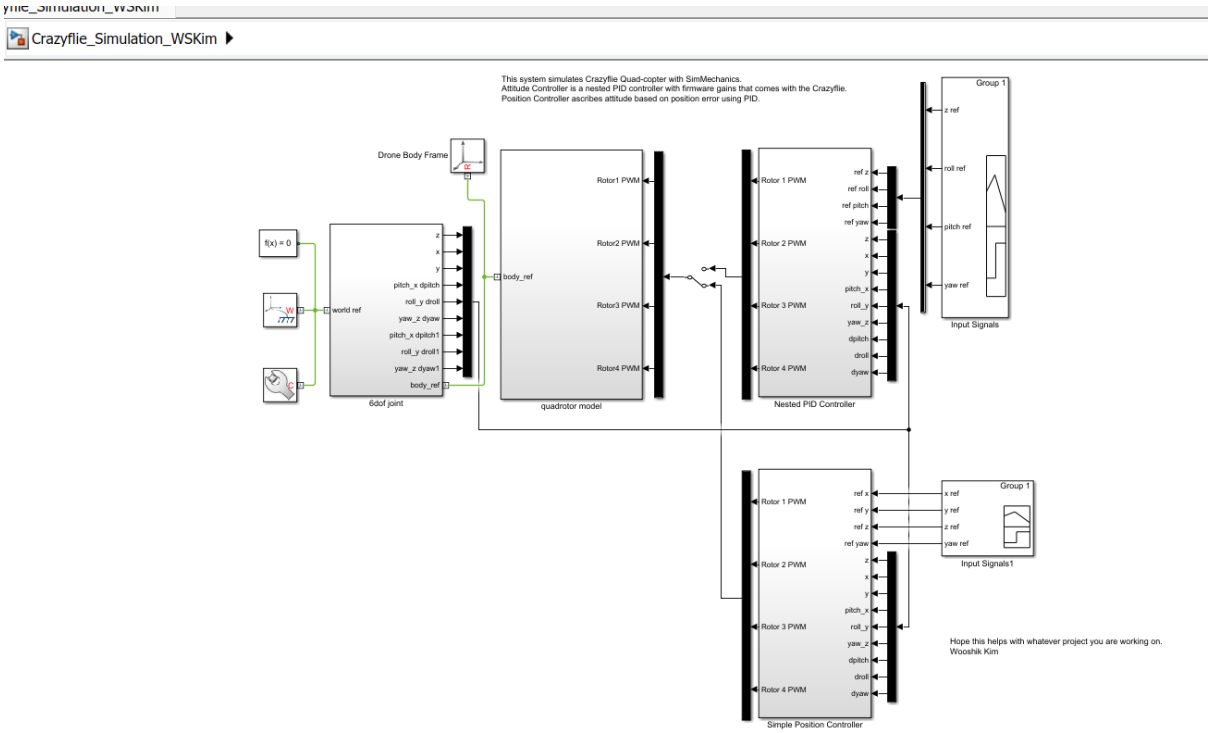
The mapping of PWM to thrust and propeller rotation aligns with the methodology outlined in the Crazyflie System Identification paper by Julian Forster.

For the position-to-attitude tracker, a straightforward PID controller is utilized to convert positional errors in the x and y axes into roll and pitch references.

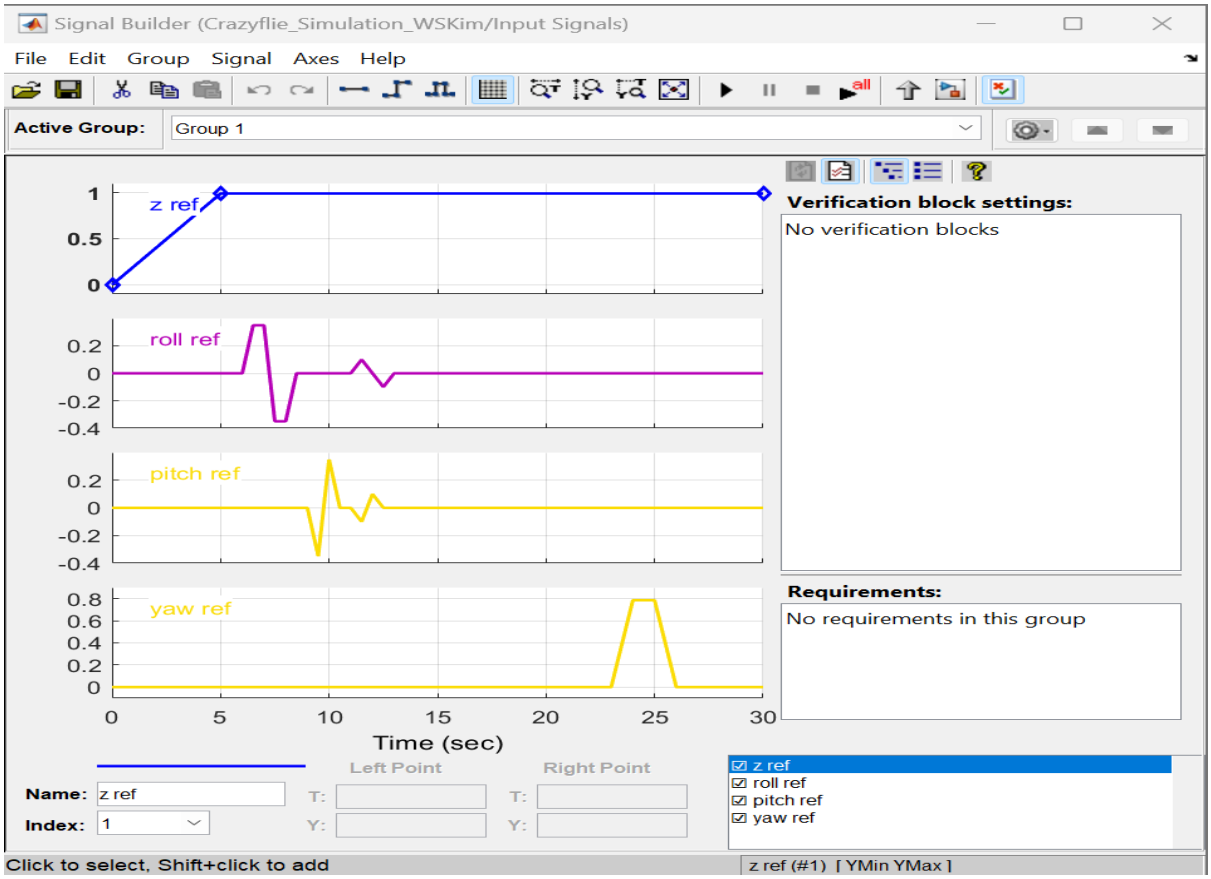
4.2 Parameters:



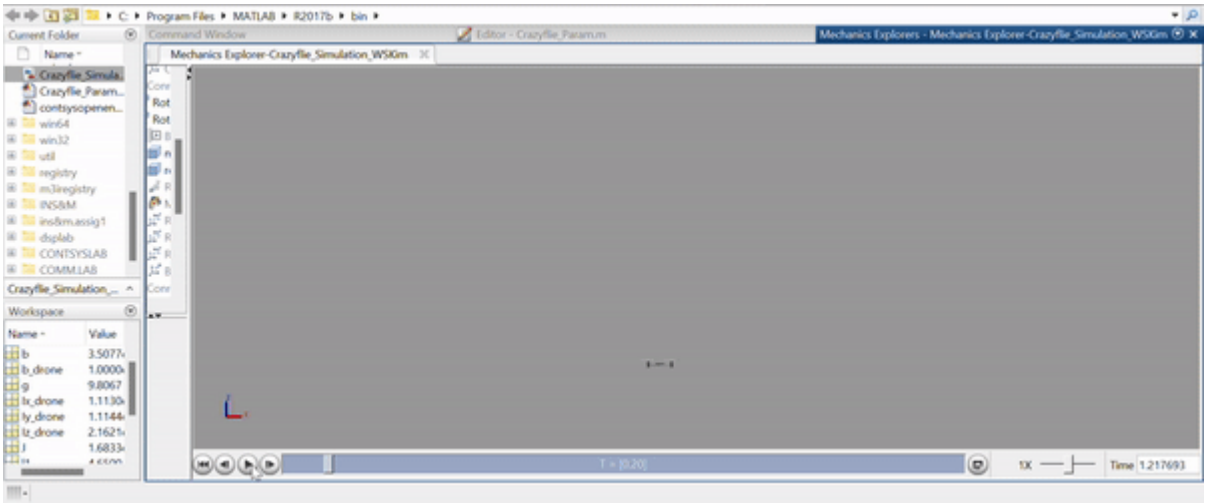
4.3 Schematic model:



4.4 Input signals



4.5 Output simulation:



5. Loco Positioning System using Anchor Node Communication

5.1 Introduction:

Loco positioning: is a local positioning system, based on Ultra-Wide Band radio that is used to find the absolute 3D position of objects in space. Thus, creating its own miniature GPS system

5.2 Explanation of working:

1. Anchors are placed in the room as reference points.
2. Tags are attached to the objects to be tracked.
3. Anchors and Tags communicate via short high frequency radio messages.
4. The system measures distances between Anchors and Tags.
5. Tag positions are calculated based on distance measurements.
6. Position calculation is done on the Tag itself (not an external computer).
7. Crazyflie 2.1 integrates its position knowledge.

5.3 Methods of loco positioning:

There are 3 different modes for setting up loco positioning:

1. **Two Way Ranging (TWR) Mode:** Tag actively pings anchors to measure distances, requiring around 6 anchors for 3D positioning. It's highly accurate, works even when a tag leaves anchor-defined space, but only positions one tag or Crazyflie at a time.
2. **Time Difference of Arrival 2 (TDoA 2) Mode:** Anchors send sync packets, and tags calculate relative distances from these packets, allowing positioning of multiple tags simultaneously. Best within anchor-defined space, needs 8 anchors for precision.
3. **Time Difference of Arrival 3 (TDoA 3) Mode:** Similar to TDoA 2 but with a randomized transmission schedule, enabling more anchors for scalability, extended range in Long Range mode, and robustness against anchor changes or losses.

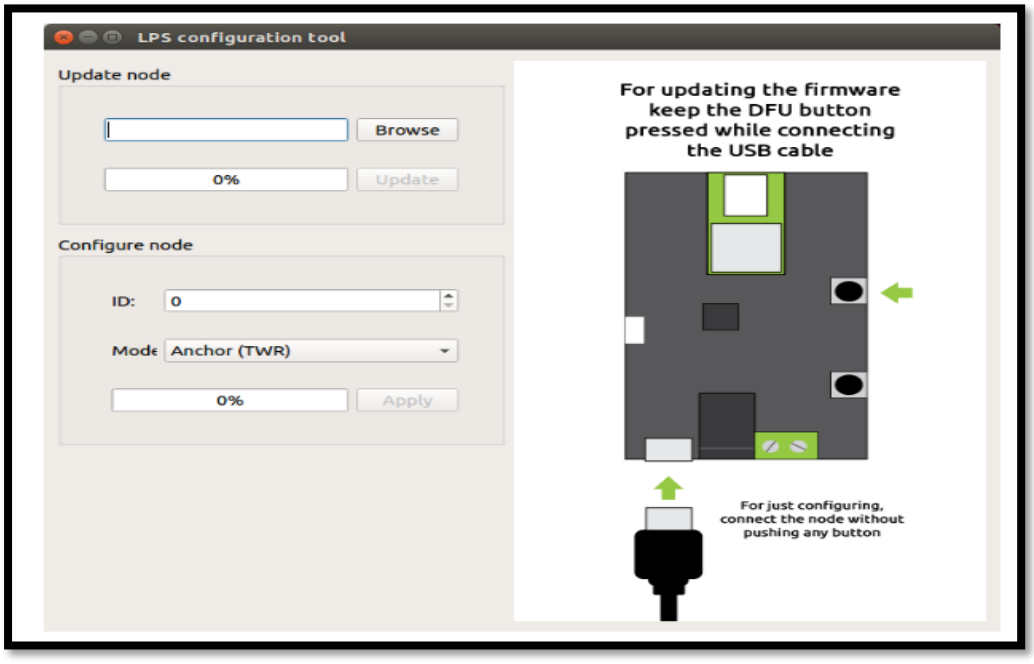
5.4 Software requirements:

1 install cfclient : https://www.bitcraze.io/documentation/repository/crazyflie-clients-python/master/installation/install/

2 install usb driver for LPS Node: https://www.bitcraze.io/documentation/repository/lps-node-firmware/master/building/zadig_lps/

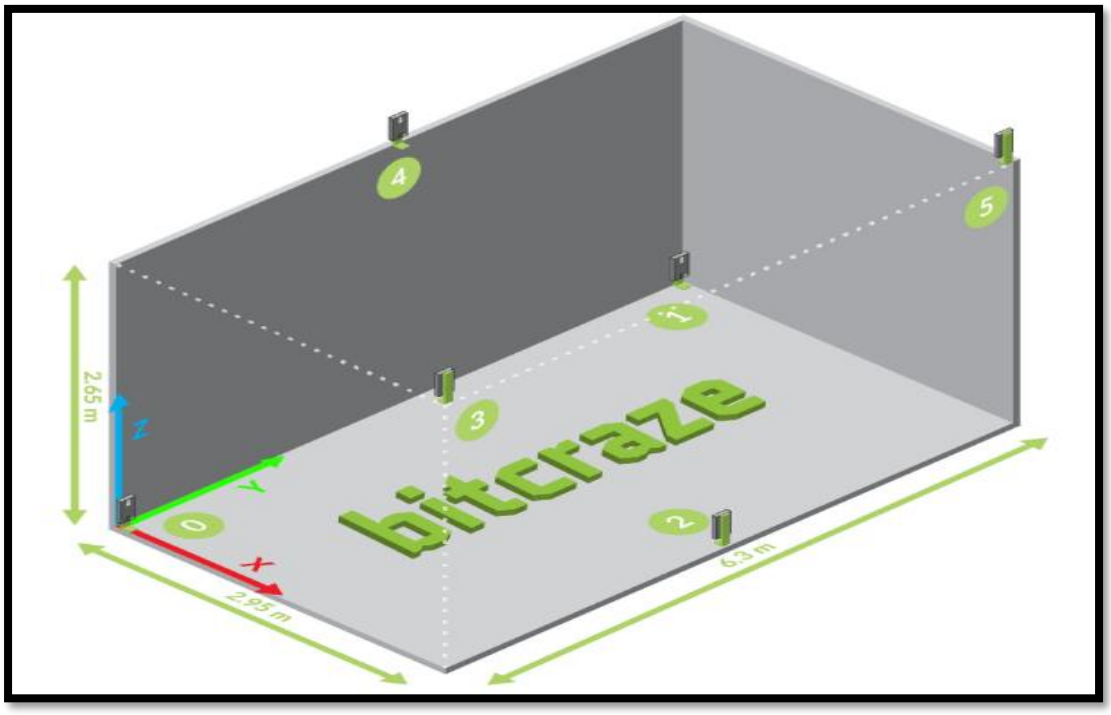
5.5 Steps for setting up system:

1. The LPS configuration tool is used for configuring the Loco Node via USB[9].



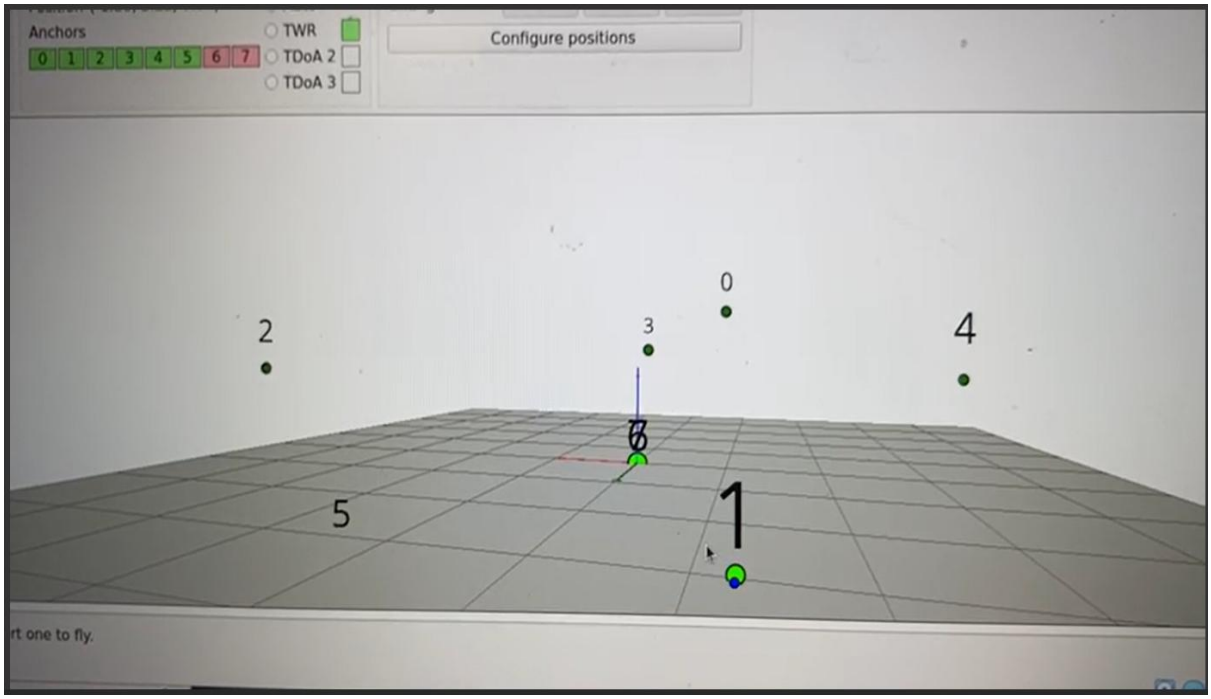
2. Use the LPS configuration tool to update the firmware and set the addresses of the nodes from 0-5. The nodes should be placed at least 15 cm away from walls or ceiling.

3. In this reference setup we have placed 3 anchors above and 3 anchors bellow the flight area, shaped in inverse triangles



5.6 Experimental results:

6 nodes were placed in a room and the crazyflie was placed on node 1 the system accurately marked its position with a blue dot representing the crazyflie's position



Drone placed next to node 1



6. Ros2 (RVIZ2) (Connections + Readings)

6.1 Software requirments:

1. Install ubuntu linux humble
2. Install ROS 2 https://docs.ros.org/en/humble/Installation.html
3. Install dependencies from https://imrclab.github.io/crazyswarm2/installation.html

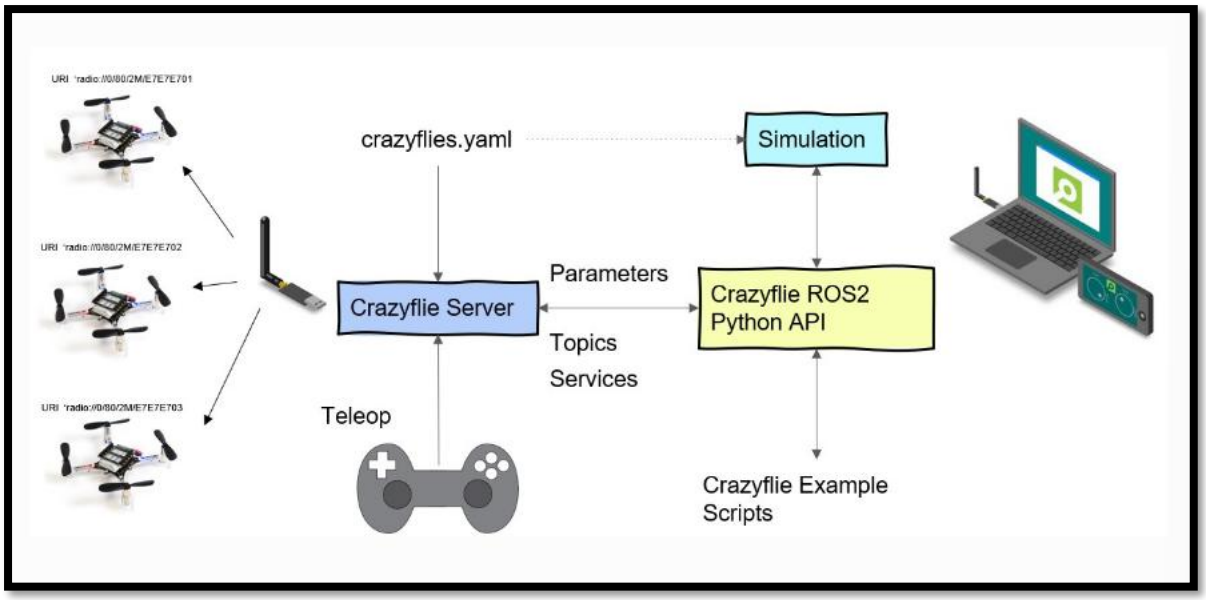
6.2 Explanation:

Crazyflie server

The crazyflie server node connects multiple Crazyflies with on or more Crazyradio PA dongles.

It has two backends that you can choose from:

- **cpp**: This is based on the crazyflie-link-cpp on the lowest layer. [7]
- **cflib**: This is based on the crazyflie-lib-python up until the logging, parameter, and command sending handling.

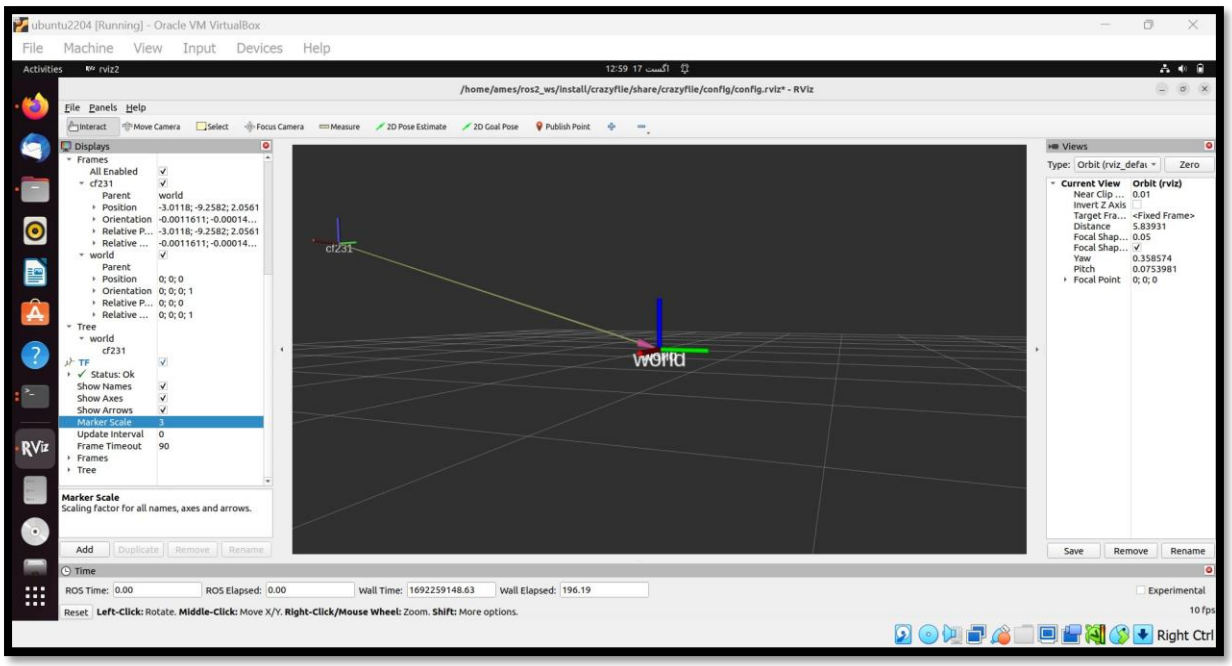


6.3 ROS 2 configurations:

All configuration files are in crazyflie/config.[8]

- crazyflies.yaml : setting up everything related to the robots.
- server.yaml : setting up everything related to the server.
- motion_capture.yaml : configs for the motion capture package.
- teleop.yaml : configs for remote controls.

Visual results of crazyflie(cf231) connected to PA radio on ROS2



6.4 Challenges:

For mapping the room with ROS2 SLAM Toolbox was not possible as the flow deck and multi-ranger deck were unavailable [9]

7. Flight control with Python API

7.1 Software installation:

To connect Crazyflie through the Crazyflie python library by a python script. This is the starting point for developing for the Crazyflie for off-board enabled flight.

1. install Python 3.8 on OS from Welcome to Python.org
2. check version of PIP3 and python use command pip3 –version
3. install cflib using command *"pip3 install cflib"*
4. *write and run code on python idle*

7.2 Connecting, Logging and Parameters:[10]

7.3 Step 1. Connecting with the crazyflie:

Overview:

this code sets up the necessary infrastructure to connect to a Crazyflie drone, defines a callback function to handle the connection event, and attempts to establish a wireless communication link with the drone using a specified URI.

Code:

```
import cflib.crtp
from cflib.crazyflie import Crazyflie

# Callback function to be executed when Crazyflie is connected
def crazyflie_connected(link_uri):

    print("Connected to Crazyflie at URI:", link_uri)

# Initialize the Crazyflie drivers
cflib.crtp.init_drivers()

# Create an instance of the Crazyflie class
crazyflie = Crazyflie()

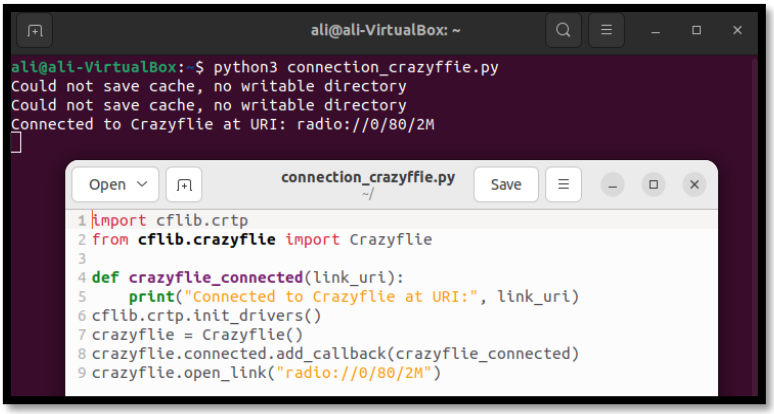
# Add the connected callback to the Crazyflie instance
crazyflie.connected.add_callback(crazyflie_connected)
```

```
# Open a communication link with the Crazyflie
# Replace "radio://0/80/2M" with the desired URI

crazyflie.open_link("radio://0/80/2M")
```

Output:

Using Python, we successfully created a connection with the Crazyflie drone. This connection is the foundation for all further interactions and control of the drone.



7.4 Step 2.Takeoff+deck_Detection:

Concept overview:

this script is used to perform a crucial check before takeoff. Specifically, we ensured that the deck (additional hardware or sensors) was properly attached to the Crazyflie drone. Once this essential check was successfully passed, our script initiated the takeoff sequence, allowing the drone to lift off and begin its flight operations. This careful scripting ensured that the drone would only take off when all necessary components were in place, enhancing safety and operational efficiency.

Code:

```
import logging
import sys
import time

from threading import Event

import cflib.crtp

from cflib.crazyflie import Crazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
```

```
from cflib.positioning.motion_commander import MotionCommander
from cflib.positioning.position_hl_commander import PositionHlCommander
from cflib.utils import uri_helper

URI = uri_helper.uri_from_env(default='radio://0/80/2M/E7E7E7E7E7')

DEFAULT_HEIGHT = 0.3

#The deck_attached_event is a global variable
deck_attached_event = Event()

#to ensure a check a deck is attached
#param_deck_flow() is a string type, so you will need to convert it to a number

def param_deck_flow(_, value_str):
value = int(value_str)
print(value)

if value:
      deck_attached_event.set()
  print('Deck is attached!')

 else:
print('Deck is NOT attached!')

def take_off_simple(scf):
with MotionCommander(scf) as mc:

 print('T1')

mc.up(DEFAULT_HEIGHT)
print('default height')
time.sleep(2)
 mc.stop()

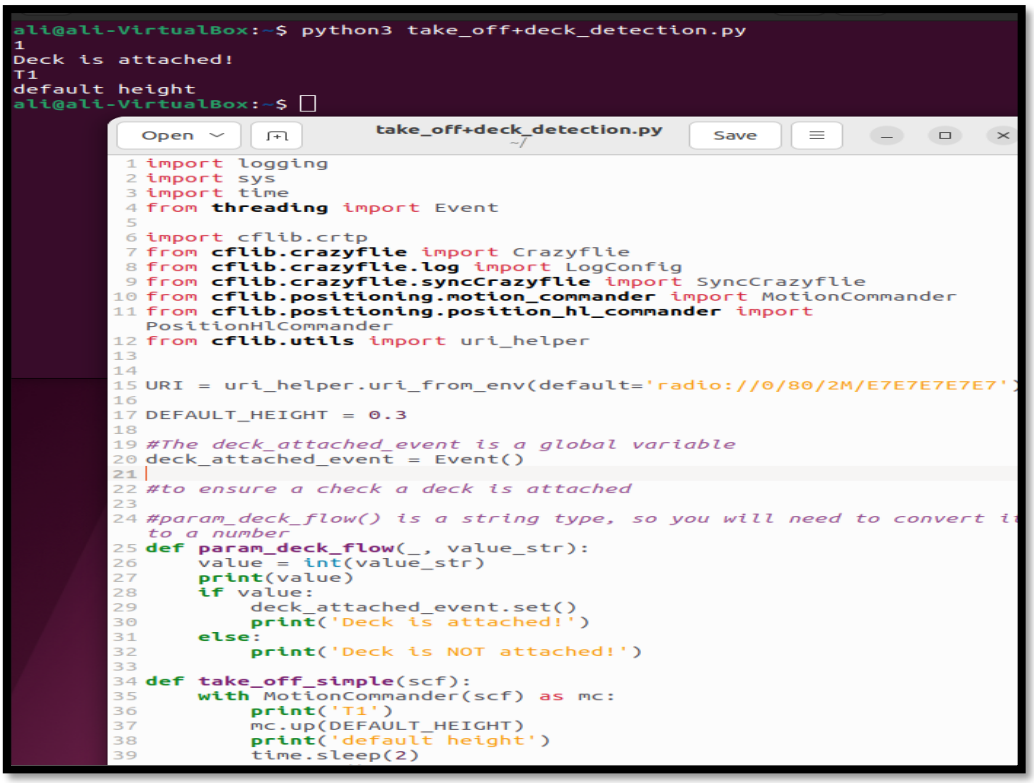
if __name__ == '__main__':
cflib.crtp.init_drivers()
with SyncCrazyflie(URI, cf=Crazyflie(rw_cache='./cache')) as scf
scf.cf.param.add_update_callback(group="deck", name="bcLoco", cb=param_deck_flow)

#Check out the deck which you are using either its bcloco or bcflow
time.sleep(1)

  if not deck_attached_event.wait(timeout=5):
# If this returns false, the function will not be called and the crazyflie will not take off.
 print('No flow deck detected!')
sys.exit(1)

 take_off_simple(scf)
```

OUTPUT:



7.5 Step 2. Logging Synchronous:

Concept overview:

The code connects to a Crazyflie quadcopter, logs its stabilization data (roll, pitch, yaw) with synchronized timing for 10 iterations, and then disconnects. The printed output displays the timestamp, log configuration name, and stabilization values, enhancing visibility for monitoring and analysis during flight testing.

Code:

```
import logging

import time
import cflib.crtp

from cflib.crazyflie import Crazyflie
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncLogger import SyncLogger
from cflib.utils import uri_helper

# URI to the Crazyflie to connect to
uri = uri_helper.uri_from_env(default='radio://0/80/2M/E7E7E7E7E7')

# Only output errors from the logging framework
logging.basicConfig(level=logging.ERROR)

def log_stab_callback(timestamp, data, logconf):
```

```
    print('[%d][%s]: Roll=%.2f | Pitch=%.2f | Yaw=%.2f' % (timestamp, logconf.name, data['stabilizer.roll'],
data['stabilizer.pitch'], data['stabilizer.yaw']))

def simple_log(scf, logconf):

    with SyncLogger(scf, logconf) as logger:
 for i, log_entry in enumerate(logger):  # Use enumerate to keep track of iterations

  timestamp, data, logconf_name = log_entry

          print('[%d][%s]: Roll=%.2f | Pitch=%.2f | Yaw=%.2f' % (timestamp, logconf_name, data['stabilizer.roll'],
data['stabilizer.pitch'], data['stabilizer.yaw']))

          if i >= 9:  # Stop after 10 iterations (0-9)
 print("Now I will disconnect :'(")
break
time.sleep(1)

          def simple_connect():
print("Yeah, I'm connected! :D")

    time.sleep(3)

    if __name__ == '__main__':
  # Initialize the low-level drivers

cflib.crtp.init_drivers()


    lg_stab = LogConfig(name='Stabilizer', period_in_ms=10 lg_stab.add_variable('stabilizer.roll', 'float')
lg_stab.add_variable('stabilizer.pitch', 'float')
lg_stab.add_variable('stabilizer.yaw', 'float')

with SyncCrazyflie(uri, cf=Crazyflie(rw_cache='./cache')) as scf:

       simple_connect()
  simple_log(scf, lg_stab)
```

Output:

```
>>>
================== RESTART: C:/Python38/logging.synchronus.py ==================
Yeah, I'm connected! :D
[13020][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.11 | Pitch=-0.48 | Yaw=0.62
[13030][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.11 | Pitch=-0.48 | Yaw=0.62
[13040][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.11 | Pitch=-0.47 | Yaw=0.62
[13050][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.11 | Pitch=-0.47 | Yaw=0.62
[13060][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.11 | Pitch=-0.47 | Yaw=0.62
[13070][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.11 | Pitch=-0.47 | Yaw=0.62
[13080][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.12 | Pitch=-0.47 | Yaw=0.62
[13090][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.12 | Pitch=-0.47 | Yaw=0.62
[13100][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.13 | Pitch=-0.46 | Yaw=0.62
[13110][<cflib.crazyflie.log.LogConfig object at 0x000001CD3179A280>]: Roll=0.13 | Pitch=-0.46 | Yaw=0.62
Now I will disconnect :'(
>>>
```

7.7 Step 3. Logging Asynchronous:

Concept overview:

This code performs asynchronous logging. By setting up asynchronous logging in this way, can receive and process logging data independently from the main control loop. This allowed us to continuously collect data from the Crazyflie while the main control loop handles flight or other tasks.

Code:

```
import logging
import time
import cflib.crtp

from cflib.crazyflie import Crazyflie
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncLogger import SyncLogger
from cflib.utils import uri_helper

# URI to the Crazyflie to connect to
uri = uri_helper.uri_from_env(default='radio://0/80/2M/E7E7E7E7E7')

# Only output errors from the logging framework
logging.basicConfig(level=logging.ERROR)

def log_stab_callback(timestamp, data, logconf):
    roll = data['stabilizer.roll']
    pitch = data['stabilizer.pitch']
    yaw = data['stabilizer.yaw']

    print('[%d][%s]: Roll=%.2f° | Pitch=%.2f° | Yaw=%.2f°' % (timestamp, logconf.name, roll, pitch, yaw))

def simple_log_async(scf, logconf):
    cf = scf.cf
    cf.log.add_config(logconf)
    logconf.data_received_cb.add_callback(log_stab_callback)
    logconf.start()

    try:
        while True:

            time.sleep(1)
    except KeyboardInterrupt:
        print("\nKeyboard interrupt detected. Disconnecting...")
    finally:

        logconf.stop()
def simple_connect():
    print("Yeah, I'm connected! :D")
    time.sleep(5)
    print("I am disconnected now.")

if __name__ == '__main__':
    # Initialize the low-level drivers
    cflib.crtp.init_drivers()
```

```
lg_stab = LogConfig(name='Stabilizer', period_in_ms=10)
lg_stab.add_variable('stabilizer.roll', 'float')
lg_stab.add_variable('stabilizer.pitch', 'float')
lg_stab.add_variable('stabilizer.yaw', 'float')

with SyncCrazyflie(uri, cf=Crazyflie(rw_cache='./cache')) as scf:
    simple_connect()
    simple_log_async(scf, lg_stab)
```

Output:

```
[9801][Stabilizer]: Roll=-0.01° | Pitch=-0.18° | Yaw=0.01°
[9811][Stabilizer]: Roll=0.00° | Pitch=-0.20° | Yaw=0.01°
[9821][Stabilizer]: Roll=0.03° | Pitch=-0.20° | Yaw=0.01°
[9831][Stabilizer]: Roll=0.06° | Pitch=-0.20° | Yaw=0.01°
[9841][Stabilizer]: Roll=0.07° | Pitch=-0.23° | Yaw=0.01°
[9851][Stabilizer]: Roll=0.07° | Pitch=-0.26° | Yaw=0.01°
[9861][Stabilizer]: Roll=0.06° | Pitch=-0.29° | Yaw=0.01°
[9871][Stabilizer]: Roll=0.03° | Pitch=-0.33° | Yaw=0.01°
[9881][Stabilizer]: Roll=0.02° | Pitch=-0.36° | Yaw=0.01°
[9891][Stabilizer]: Roll=0.01° | Pitch=-0.38° | Yaw=0.01°
[9901][Stabilizer]: Roll=0.01° | Pitch=-0.39° | Yaw=0.01°
[9911][Stabilizer]: Roll=0.00° | Pitch=-0.39° | Yaw=0.01°
[9921][Stabilizer]: Roll=0.01° | Pitch=-0.38° | Yaw=0.01°
[9931][Stabilizer]: Roll=0.01° | Pitch=-0.36° | Yaw=0.01°
[9941][Stabilizer]: Roll=0.02° | Pitch=-0.33° | Yaw=0.01°
[9951][Stabilizer]: Roll=0.04° | Pitch=-0.30° | Yaw=0.01°
[9961][Stabilizer]: Roll=0.04° | Pitch=-0.26° | Yaw=0.01°
[9971][Stabilizer]: Roll=0.04° | Pitch=-0.25° | Yaw=0.01°
[9981][Stabilizer]: Roll=0.02° | Pitch=-0.24° | Yaw=0.01°
[9991][Stabilizer]: Roll=0.01° | Pitch=-0.24° | Yaw=0.01°
[10001][Stabilizer]: Roll=0.01° | Pitch=-0.23° | Yaw=0.01°
[10011][Stabilizer]: Roll=0.01° | Pitch=-0.23° | Yaw=0.00°
[10021][Stabilizer]: Roll=0.01° | Pitch=-0.23° | Yaw=0.00°
[10031][Stabilizer]: Roll=-0.01° | Pitch=-0.23° | Yaw=0.01°
[10041][Stabilizer]: Roll=-0.02° | Pitch=-0.22° | Yaw=0.00°
[10051][Stabilizer]: Roll=-0.01° | Pitch=-0.19° | Yaw=0.01°
[10061][Stabilizer]: Roll=-0.01° | Pitch=-0.18° | Yaw=0.01°
[10071][Stabilizer]: Roll=-0.02° | Pitch=-0.17° | Yaw=0.01°
[10081][Stabilizer]: Roll=-0.03° | Pitch=-0.17° | Yaw=0.01°
[10091][Stabilizer]: Roll=-0.05° | Pitch=-0.15° | Yaw=0.01°
[10101][Stabilizer]: Roll=-0.07° | Pitch=-0.16° | Yaw=0.01°
[10111][Stabilizer]: Roll=-0.08° | Pitch=-0.15° | Yaw=0.00°
[10121][Stabilizer]: Roll=-0.07° | Pitch=-0.14° | Yaw=0.01°
[10131][Stabilizer]: Roll=-0.04° | Pitch=-0.10° | Yaw=0.01°

Keyboard interrupt detected. Disconnecting...[10141][Stabilizer]: Roll=-0.03° | Pitch=-0.09° | Yaw=0.01°
```

7.8 Step 4. Parameter retrieve +update:

Concept overview:

This code is used for accessing and modifying preset parameters of the Crazyflie drone through Python. This step allows us to customize various aspects of the drone's behavior and performance according to our project requirements.

Code:

```
import logging

import time
import cflib.crtp

from cflib.crazyflie import Crazyflie
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncLogger import SyncLogger
from cflib.utils import uri_helper
```

```
\# URI to the Crazyflie to connect to
uri = uri_helper.uri_from_env(default='radio://0/80/2M/E7E7E7E7E7')

def param_stab_est_callback(name, value):
    print('Parameter update - Name: {}, New Value: {}'.format(name, value))

def simple_param_async(scf, groupstr, namestr):
    cf = scf.cf
    full_name = groupstr + "." + namestr
   print('Getting initial parameter value for: ' + full_name)

    initial_value = cf.param.get_value(full_name)
    print('Initial value:', initial_value)
    time.sleep(5)

    cf.param.add_update_callback(group=groupstr, name=namestr,
cb=param_stab_est_callback)

    time.sleep(1)
    new_value = 1
    cf.param.set_value(full_name, new_value)
    print('Setting parameter {} to new value: {}'.format(full_name, new_value))    time.sleep(1)

    updated_value = cf.param.get_value(full_name)
 if updated_value != initial_value
        print('updated')

    else:
print('No parameter update detected.')
    print('End of simple_param_async function.')

def simple_connect():
    print("Connected to Crazyflie! :D")
    time.sleep(3)
   if __name__ == '__main__':

    # Initialize the low-level drivers
 cflib.crtp.init_drivers()

        # Adding parameter variables
  group = "stabilizer"
    name = "estimator"

    with SyncCrazyflie(uri, cf=Crazyflie(rw_cache='./cache')) as scf:
        simple_connect()
        simple_param_async(scf, group, name)
```

Output:

```
====================== RESTART: C:/Python38/parameters.py ======================
Connected to Crazyflie! :D
Getting initial parameter value for: stabilizer.estimator
Initial value: 2
Setting parameter stabilizer.estimator to new value: 1
Parameter update - Name: stabilizer.estimator, New Value: 1
updated
End of simple_param_async function.
>>>
```

7.9 Step 5. Flying + Motion Commander + Loggings + Graphs:

Concept overview:

The main part of the code initializes Crazyflie drivers, establishes a connection, configures the log for position data, and waits for the deck attachment event. If the deck is not attached, the script exits. Otherwise, it starts logging, takes off, and stops logging after flight. After the flight, the code plots the recorded position data for the drone's x, y, and z coordinates over time using Matplotlib.

Code:

```
import logging
import sys
import time
from threading import Event

import cflib.crtp
from cflib.crazyflie import Crazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
from cflib.positioning.motion_commander import MotionCommander
from cflib.positioning.position_hl_commander import PositionHlCommander
from cflib.utils import uri_helper
import matplotlib.pyplot as plt

URI = uri_helper.uri_from_env(default='radio://0/80/2M/E7E7E7E7E7')

DEFAULT_HEIGHT = 0.3

velocity=0.2
deck_attached_event = Event()

logging.basicConfig(level=logging.ERROR)

position_estimate = [0, 0, 0]

t = []
x = []
z = []

y = []
i = 0

for _ in range(1000):

    x.append(0)

for _ in range(1000):

    y.append(0)

for _ in range(1000):

    t.append(0)

for _ in range(1000):
```

```
    z.append(0)

#param_deck_flow() is a string type, so you will need to convert it to a number

def param_deck_flow(_, value_str):
value = int(value_str)
    print(value)
    if value:
        deck_attached_event.set()
        print('Deck is attached!')
    else:
        print('Deck is NOT attached!')

def take_off_simple(scf):
 print('T2')
    with MotionCommander(scf, default_height = DEFAULT_HEIGHT) as mc:
        print('T3')

            time.sleep(2)
 mc.stop()

def log_pos_callback(timestamp, data, logconf):

    print(data)
 global position_estimate
    global x
    global y
    global i
    global z

    position_estimate[0] = data['stateEstimate.x']
position_estimate[1] = data['stateEstimate.y']
    position_estimate[2] = data['stateEstimate.z']

    if i < 1000:  # Ensure i does not exceed the maximum index
    i = i + 1
        t[i] = i

        x[i] = position_estimate[0]
        y[i] = position_estimate[1]
        z[i] = position_estimate[2]

if __name__ == '__main__':
    cflib.crtp.init_drivers()
    with SyncCrazyflie(URI, cf=Crazyflie(rw_cache='./cache')) as scf:



        logconf = LogConfig(name='Position', period_in_ms=10)
        logconf.add_variable('stateEstimate.x', 'float')
        logconf.add_variable('stateEstimate.y', 'float')
        logconf.add_variable('stateEstimate.z', 'float')
        scf.cf.log.add_config(logconf)
        logconf.data_received_cb.add_callback(log_pos_callback)


        scf.cf.param.add_update_callback(group="deck", name="bcLoco", cb=param_deck_flow)
        time.sleep(1)

        if not deck_attached_event.wait(timeout=5):# If this returns false, the function will not be called and the
crazyflie will not take off.
```

```
        print('No flow deck detected!')

        sys.exit(1)
    print('T1')

    logconf.start()

    take_off_simple(scf)

    logconf.stop()

    plt.figure(0)
plt.plot(t, x, marker='o')
    plt.xlabel('time')
    plt.ylabel('x Position')
    plt.title('x position vs time')
    plt.xlim(250, 1000)
    plt.grid()

        plt.figure(1)
    plt.plot(t, y, marker='o')
    plt.xlabel('time')
    plt.ylabel('Y Position')
    plt.title('y position vs time ')
    plt.xlim(250, 1000)
    plt.grid()

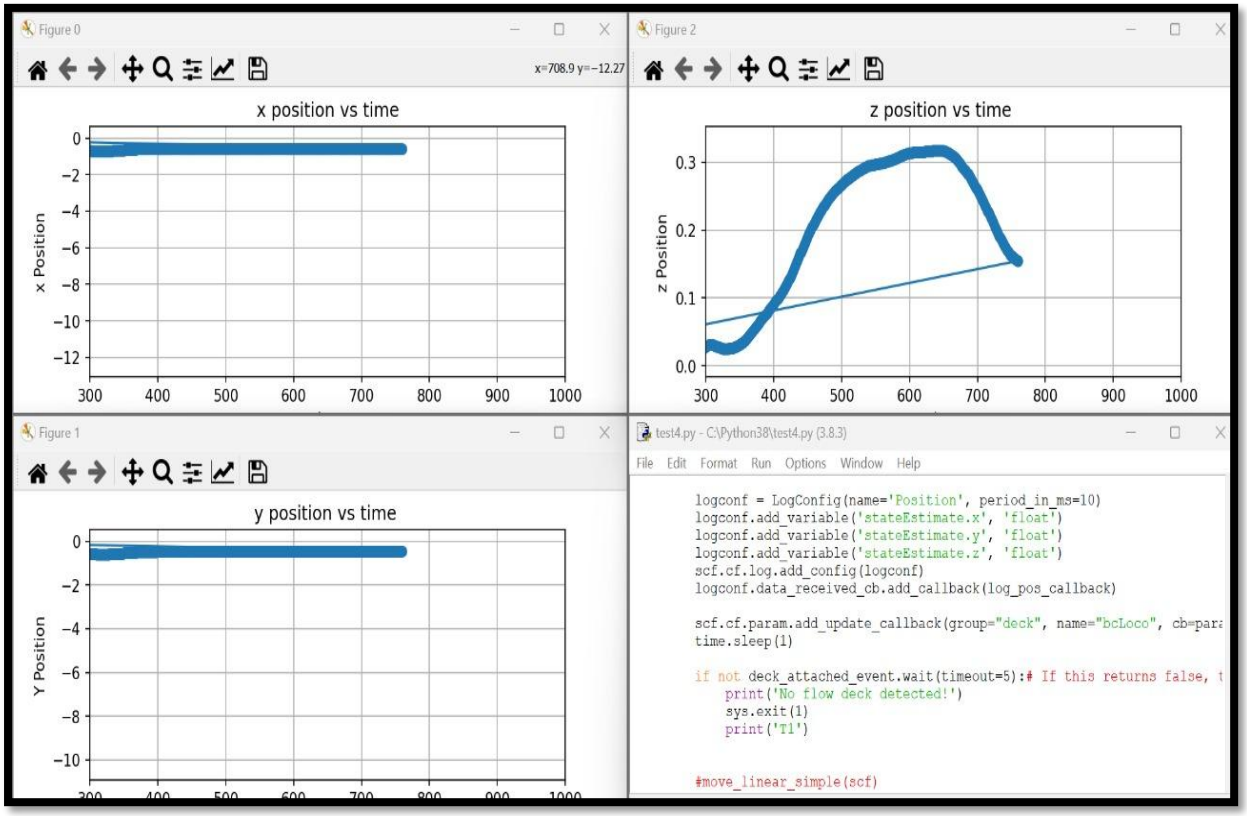
    plt.figure(2)
    plt.plot(t, z, marker='o')
    plt.xlabel('time')
    plt.ylabel('z Position')
    plt.title('z position vs time ')
    plt.grid()
    plt.xlim(250, 1000)
    plt.show()
```

OUTPUT:

X,Y,Z position logging on monitor

```
'stateEstimate.x': 0.15111815929412842, 'stateEstimate.y': 0.17669735848903656, 'stateEstimate.z': 0.3288229703903198}
'stateEstimate.x': 0.15122951567173004, 'stateEstimate.y': 0.17665542662143707, 'stateEstimate.z': 0.3282734751701355}
'stateEstimate.x': 0.15134553611278534, 'stateEstimate.y': 0.17661193013191223, 'stateEstimate.z': 0.3277082145214081}
'stateEstimate.x': 0.15146589279174805, 'stateEstimate.y': 0.17656677961349487, 'stateEstimate.z': 0.32712075114250183}
'stateEstimate.x': 0.15159018337726593, 'stateEstimate.y': 0.17651991546154022, 'stateEstimate.z': 0.3264872133731842}
'stateEstimate.x': 0.1517181098461151, 'stateEstimate.y': 0.17647142708301544, 'stateEstimate.z': 0.32579100131988525}
'stateEstimate.x': 0.15184946358203888, 'stateEstimate.y': 0.17642147839069366, 'stateEstimate.z': 0.3250240981578827}
'stateEstimate.x': 0.15198419988155365, 'stateEstimate.y': 0.17637021839618683, 'stateEstimate.z': 0.32418894767761123}
'stateEstimate.x': 0.1521223783493042, 'stateEstimate.y': 0.17631788551807404, 'stateEstimate.z': 0.3233029842376709}
'stateEstimate.x': 0.15226390957832336, 'stateEstimate.y': 0.17626476287841797, 'stateEstimate.z': 0.3223753273487091}
'stateEstimate.x': 0.1524086445569992, 'stateEstimate.y': 0.1762111485004425, 'stateEstimate.z': 0.3214040696620941}
'stateEstimate.x': 0.15255653858184814, 'stateEstimate.y': 0.17615719139575958, 'stateEstimate.z': 0.3203955590724945}
'stateEstimate.x': 0.1527075469493866, 'stateEstimate.y': 0.17610293626785278, 'stateEstimate.z': 0.31935638189315796}
'stateEstimate.x': 0.152861550450325, 'stateEstimate.y': 0.17604833841323853, 'stateEstimate.z': 0.31829002499580383}
'stateEstimate.x': 0.1530182957649231, 'stateEstimate.y': 0.17599327862262726, 'stateEstimate.z': 0.3171919882297516}
'stateEstimate.x': 0.15317754447460175, 'stateEstimate.y': 0.17593753337860107, 'stateEstimate.z': 0.3160516023635864}
'stateEstimate.x': 0.15333904325962067, 'stateEstimate.y': 0.17588073015213013, 'stateEstimate.z': 0.3148707449436188}
'stateEstimate.x': 0.15350250899791718, 'stateEstimate.y': 0.17582249641418457, 'stateEstimate.z': 0.31364792585372925}
'stateEstimate.x': 0.15366770327091217, 'stateEstimate.y': 0.17576254904270172, 'stateEstimate.z': 0.3123769462108612}
'stateEstimate.x': 0.15383420884609222, 'stateEstimate.y': 0.1757005751132965, 'stateEstimate.z': 0.31106236577033997}
'stateEstimate.x': 0.15400129556655884, 'stateEstimate.y': 0.17563633620738983, 'stateEstimate.z': 0.30971914529800415}
'stateEstimate.x': 0.15416809916496277, 'stateEstimate.y': 0.17556972801685333, 'stateEstimate.z': 0.30835312604904175}
'stateEstimate.x': 0.15433387458324432, 'stateEstimate.y': 0.17550061643123627, 'stateEstimate.z': 0.30696260929107666}
'stateEstimate.x': 0.1544979214668274, 'stateEstimate.y': 0.17542895674705505, 'stateEstimate.z': 0.30555182695388794}
'stateEstimate.x': 0.15465950965881348, 'stateEstimate.y': 0.17535483837127686, 'stateEstimate.z': 0.30411726236343384}
'stateEstimate.x': 0.15481801331043243, 'stateEstimate.y': 0.17527851462364197, 'stateEstimate.z': 0.30265340209007263}
'stateEstimate.x': 0.1549728661775589, 'stateEstimate.y': 0.17520029842853546, 'stateEstimate.z': 0.301161319017403}
'stateEstimate.x': 0.15512363612651825, 'stateEstimate.y': 0.17512072622776031, 'stateEstimate.z': 0.2996392548084259}
'stateEstimate.x': 0.15527011454105377, 'stateEstimate.y': 0.17504040896892548, 'stateEstimate.z': 0.2980739176273346}
'stateEstimate.x': 0.15541231632232666, 'stateEstimate.y': 0.17495989799499512, 'stateEstimate.z': 0.29645276069641113}
'stateEstimate.x': 0.15555037558078766, 'stateEstimate.y': 0.1748797744512558, 'stateEstimate.z': 0.2947733104228973}
```

Graphical output of the drone taking off then hovering for a few seconds then landing:



7.10 Step 6. Flying in a box + Motion Commander + Loggings Graphs:

Concept overview

The script connects to a Crazyflie drone, performs controlled movements within specified limits, logs position-related data, and visualizes the collected data using Matplotlib.

Code:

```
import logging
import sys
import time

from threading import Event

import cflib.crtp
from cflib.crazyflie import Crazyflie
from cflib.crazyflie.log import LogConfig
from cflib.crazyflie.syncCrazyflie import SyncCrazyflie
from cflib.positioning.motion_commander import MotionCommander
from cflib.positioning.position_hl_commander import PositionHlCommander
from cflib.utils import uri_helper

import matplotlib.pyplot as plt
```

```
URI = uri_helper.uri_from_env(default='radio://0/80/2M/E7E7E7E7E7')


DEFAULT_HEIGHT = 0.3
BOX_LIMIT = 0.2
velocity=0.2

deck_attached_event = Event()
logging.basicConfig(level=logging.ERROR)
position_estimate = [0, 0]

l = []
t = []
x = []
z = []
temp1 = []
temp2 = []
temp3 = []
y = []
i = 0
for _ in range(3000):
    x.append(0)

for _ in range(3000):
   y.append(0)

for _ in range(3000):
    t.append(0)

for _ in range(3000):
    z.append(0)

for _ in range(300):
    l.append(0)

for _ in range(300):
    temp1.append(0)

for _ in range(300):
    temp2.append(0)

for _ in range(300):
    temp3.append(0)

def param_deck_flow(_, value_str):
    value = int(value_str)
    print(value)
    if value:

        deck_attached_event.set()
        print('Deck is attached!')

    else:
        print('Deck is NOT attached!')

def move_box_limit(scf):
    with MotionCommander(scf, default_height=DEFAULT_HEIGHT) as mc:

        print('t0')
        body_x_cmd = 0.2
```

```
        body_y_cmd = 0.1
        max_vel = 0.2

              for l in range(0, 70, +1):

           if position_estimate[0] > BOX_LIMIT
         body_x_cmd=-max_vel
               print('back')

           elif position_estimate[0] < -BOX_LIMIT:
              body_x_cmd=max_vel
              print('forward')

           if position_estimate[1] > BOX_LIMIT:
              body_y_cmd=-max_vel
              print('left')

           elif position_estimate[1] < -BOX_LIMIT:
              body_y_cmd=max_vel
              print('right')

           mc.start_linear_motion(body_x_cmd, body_y_cmd, 0)
           time.sleep(0.1)

           print(l)
           temp1[l]= position_estimate[0]
           temp3[l]= position_estimate[1]
           temp2[l]=l

        mc.start_down()
        time.sleep(1)
        mc.stop()

def log_pos_callback(timestamp, data, logconf):

    print(data)

    global position_estimate
    global x
    global y
    global i
    global z

    position_estimate[0] = data['stateEstimate.x']
    position_estimate[1] = data['stateEstimate.y']
    position_estimate[2] = data['stateEstimate.z']

    i= i+1
    t[i]=i

    x[i]=position_estimate[0]
    y[i]=position_estimate[1]
    z[i]=position_estimate[2]

    return

if __name__ == '__main__':
    cflib.crtp.init_drivers()
    with SyncCrazyflie(URI, cf=Crazyflie(rw_cache='./cache')) as scf:

   logconf = LogConfig(name='Position', period_in_ms=10)
    logconf.add_variable('stateEstimate.x', 'float'
```

```
        logconf.add_variable('stateEstimate.y', 'float')
        logconf.add_variable('stateEstimate.z', 'float')
        scf.cf.log.add_config(logconf)
        logconf.data_received_cb.add_callback(log_pos_callback)

        scf.cf.param.add_update_callback(group="deck", name="bcLoco", cb=param_deck_flow)
    time.sleep(1)

        if not deck_attached_event.wait(timeout=5):# If this returns false, the function will not be called and the crazyflie will not take off.

            print('No flow deck detected!')
        sys.exit(1)
            print('T1')

        logconf.start()
        move_box_limit(scf)
        logconf.stop()



        plt.figure(0)
        plt.plot(t, x, marker='o')
        plt.xlabel('time')
        plt.ylabel('x Position')
        plt.title('x position vs time')
        plt.xlim([200,1000])
        plt.ylim([-2,2])
        plt.grid()

        plt.figure(1)
        plt.plot(t, y, marker='o')
        plt.xlabel('time')
        plt.ylabel('Y Position')
        plt.title('y position vs time ')
        plt.xlim([200,1000])
        plt.ylim([-2,2])
        plt.grid()

        plt.figure(2)
        plt.plot(temp2, temp1, marker='o')
        plt.xlabel('unit')
        plt.ylabel('X Position')
        plt.title('X position vs unit ')
        plt.xlim([0,100])
        plt.grid()

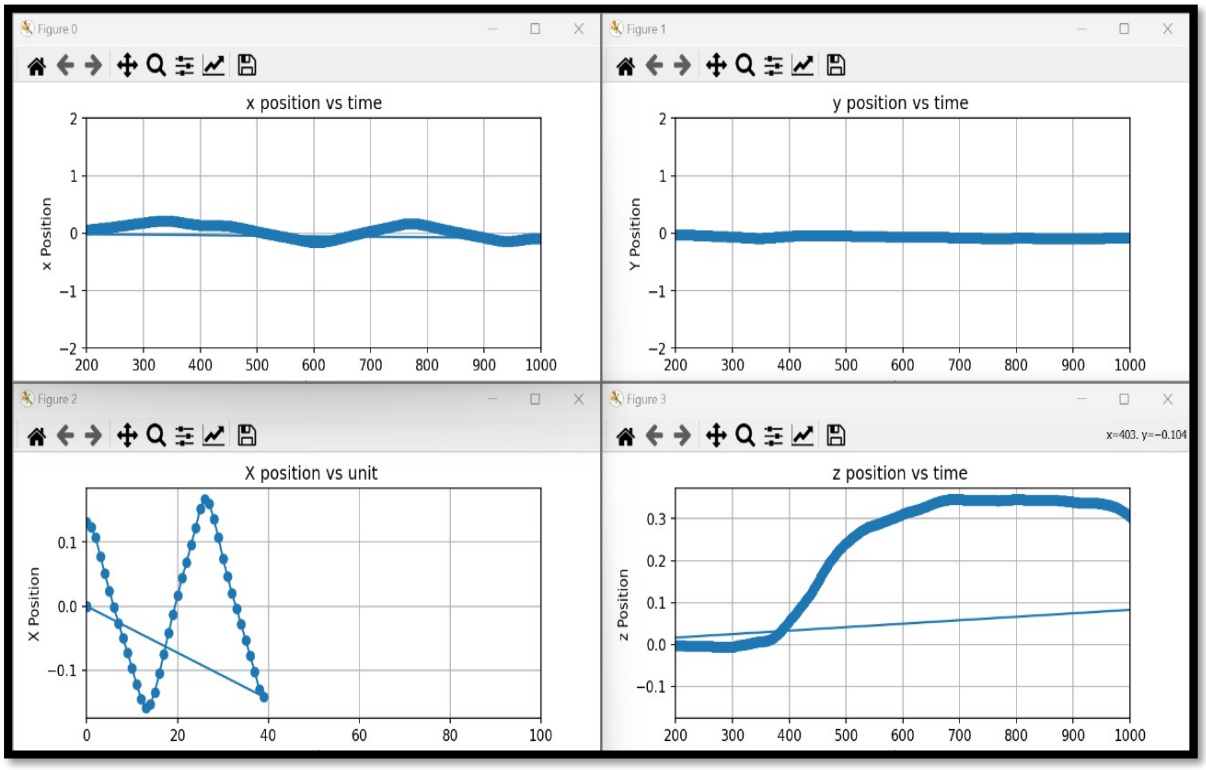
        plt.figure(3)


      plt.plot(temp2, temp3, marker='o')
        plt.xlabel('unit')
        plt.ylabel('y Position')
        plt.title('y position vs unit ')
        plt.xlim([0,100])
        plt.grid()

        plt.figure(4)
        plt.plot(t, z, marker='o')
        plt.xlabel('time')
```

```
plt.ylabel('z Position')
plt.title('z position vs time ')
plt.xlim([400,1000])
plt.ylim([-1,1])
plt.grid()
plt.show()
```

Graphical output of the drone moving in a square path:



References

[1] https://www.bitcraze.io/products/crazyflie-2-1/

[2] https://www.bitcraze.io/documentation/tutorials/getting-started-with-crazyflie-2-x/

[3] https://www.mathworks.com/matlabcentral/fileexchange/65469-crazyflie-quadcopter-simulation-using-simmechanics

[4] https://play.google.com/store/apps/details?id=se.bitcraze.crazyfliecontrol2

[5] https://www.bitcraze.io/documentation/repository/crazyflie-clients-python/master/installation/install/

[6] **https://www.bitcraze.io/documentation/repository/crazyflie-clients-python/master/userguides/userguide_client/**

[7] https://imrclab.github.io/crazyswarm2/overview.html

[8] https://imrclab.github.io/crazyswarm2/usage.html

[9] https://www.bitcraze.io/documentation/repository/lps-node-firmware/master/user-guides/configure-mode/

[10] https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/user-guides/sbs_connect_log_param/